DepthStream: Training-Free Amortized Streaming for Real-Time, Temporally Consistent Video Depth

Anonymous (draft v0.1)

Code and evaluation: `eval_sintel_v2.py` / `eval_bonn.py` / `bench_cpu.py`

Abstract

Geometric foundation models have made monocular depth estimation accurate and zero-shot general, yet the video setting exposes two costs that device-bound applications cannot pay: per-frame inference with large models is slow, and the resulting depth flickers over time. Recent propagation-based systems address both issues by *learning* a correction module around a foundation model, inheriting a training pipeline that costs GPU-days and a runtime that still targets flagship GPUs. We show that for tiny *distilled* students this learning is unnecessary. We present DepthStream, a completely training-free amortized streaming runtime: a frozen 6 M-parameter student (DepthART MobileNetV4-slim, TensorRT fp16) runs only on keyframes; in-between frames reuse the previous full-resolution depth field through backward optical flow; every re-anchor is affine-aligned to the propagated field, and a photometric-residual monitor triggers early re-anchoring exactly where flow fails. On Sintel (23 sequences) the stream at keyframe interval 3 matches per-frame inference in accuracy (δ_1 identical at 0.4614), improves ground-truth-flow temporal warping error by 15%, and cuts latency by 20%; at interval 10 with adaptive scheduling it retains 90% accuracy with a temporal error $1.75\times$ lower than a per-frame Depth Anything V2-Large teacher that is $6.7\times$ slower. Accuracy is backend-invariant (TensorRT fp16 GPU vs. ONNX-Runtime fp32 CPU: δ_1 difference $\leq 10^{-4}$), and the same runtime reaches 249 FPS end-to-end on a laptop CPU. We further contribute a failure analysis showing that speculative cascades have no headroom for distilled students, and negative results — temporal EMA filtering hurts, and flow quality is not the bottleneck — that bound the design space of training-free propagation.

1 Introduction

Monocular depth estimation has become a commodity primitive. Foundation-scale models [2, 10, 20, 21] produce accurate, affine-invariant depth for a single image, and distilled variants [1, 22] compress them to a few million parameters that fit the latency budget of device-bound applications: robotics perception, AR overlays, computational photography, and video editing.

Video breaks this equilibrium in two ways. First, *throughput*: running a foundation model on every frame of a stream is slow — our reference teacher, Depth Anything V2-Large [2], needs 56 ms per frame on an RTX 3060 Laptop GPU at 1024×436, and 26 FPS on an RTX 3090 at half-megapixel resolution is the state of the art for propagation-based systems [3]. Second, *temporal stability*: independently inferred frames flicker, because each prediction carries its own affine ambiguity and boundary noise. Video applications — segmentation, tracking, relighting, 3D capture — inherit this jitter and pay for it downstream.

A natural remedy is to stop solving every frame from scratch. Depth fields change slowly under camera and object motion: most of the signal at frame t is already present in the depth of frame $t-1$, displaced by optical flow. VeloDepth [3] recently turned this observation into a strong system: a *learned* 55 M-parameter propagation module corrects flow-warped depth and features of a frozen UniDepthV2 [21] base. The correction module is trained for six days on $8\times$RTX 4090, and the system targets flagship GPUs.

This paper starts from a different question: *for tiny distilled depth models aimed at commodity devices, is any learning needed at all?* We answer negatively. We present DepthStream, a training-free amortized streaming runtime built around a frozen 6 M-parameter DepthART student [1]. The runtime has three components (Fig. 1): (i) *keyframe inference* — the student runs every K frames; (ii) *flow-based propagation* — in-between frames reuse the previous full-resolution depth through a backward warping with DIS optical flow [24]; and (iii) *affine re-anchoring* — at every keyframe the fresh prediction is aligned to the propagated field by a closed-form scale-and-shift fit, which transfers temporal continuity to the new frame without suppressing its detail. A photometric-residual monitor closes the loop: where warping explains the observed frames poorly (occlusion, fast motion), a keyframe is triggered early.

The design is deliberately minimal, and the interesting question is what it costs and what it buys. Our evaluation on Sintel [26] (23 sequences, 1,064 frames with ground-truth depth and flow) and the real-world Bonn RGB-D Dynamic dataset [27] answers with four findings:

- **Amortization is free at moderate intervals.** At $K{=}3$, streamed accuracy equals per-frame accuracy on both datasets (δ_1 0.4614 on Sintel, 0.9637 on Bonn), temporal warping error drops 15%, and latency drops 20%. The per-frame baseline is strictly dominated.
- **Streaming beats the teacher's stability.** At $1/6.7$ of its latency, kf3 keeps 93.6% (Sintel) and 98.2% (Bonn) of

the Depth-Anything-V2-Large teacher's accuracy, with $1.37\times$ lower temporal error; adaptive scheduling at $K{=}10$ raises the consistency advantage to $1.75\times$.

- **Two intuitive fixes are harmful or pointless.** A temporal EMA on the output reduces accuracy (it re-injects stale content at keyframes); upgrading the flow (higher resolution, or ground-truth forward flow) does not improve propagation — the backward-estimated flow is already the right correspondence, and the bottleneck is occlusion structure, not flow quality.

- **Precision is backend-invariant and CPU deployment is real-time.** TensorRT fp16 on GPU and ONNX-Runtime fp32 on CPU agree to $\delta_1 \leq 10^{-4}$ over all configurations, and amortization flips the deployment calculus: the end-to-end stream reaches 249 FPS on a laptop CPU, because the model's cost is amortized to a tenth of the per-frame budget.

We additionally report two negative results that bound the design space. First, a speculative-cascade variant — running a large teacher only on disagreement regions — has no headroom for distilled students, because the student already agrees with the teacher on 93.6–98.2% of pixels; we argue this explains why propagation-based systems built on *non-distilled* foundation models [3] need learned corrections. Second, a homography-based geometric propagation variant helps only on rotation-dominated sequences and loses to dense backward flow overall — dense flow implicitly covers the full motion, and we do not find a free lunch in geometric decomposition.

Contributions.

1. A completely training-free amortized depth streaming runtime (keyframe inference, backward-flow propagation, affine re-anchoring, photometric adaptive scheduling) for tiny monocular depth models, reaching 142–176 FPS end-to-end on a 6 GB laptop GPU and 249 FPS on a CPU.

2. A dual-dataset evaluation (Sintel, Bonn) with ground-truth depth showing that moderate-interval streaming strictly dominates per-frame inference, and that the stream is 1.37–$1.75\times$ more temporally consistent than a per-frame teacher at a fraction of its cost.

3. An ablation-based analysis of the design space — EMA filtering, flow resolution, flow oracle, adaptive scheduling — and a failure analysis of speculative cascades for distilled students.

4. A reproducibility package: every table in this paper is produced by a single command; accuracy is shown to be backend-invariant across TensorRT-fp16 GPU and ONNX-Runtime-fp32 CPU.

2 Related Work

2.1 Single-image depth estimation: from relative to metric

MiDaS [10] established zero-shot *relative* depth by training on mixed datasets, and DPT [28] supplied the dense transformer decoder used by most successors. Depth Anything v1/v2 [11, 2] scaled the recipe with 62 M unlabeled images and became the de-facto open teacher; DAv2-L is the teacher in all our comparisons. A parallel line makes depth *metric*: ZoeDepth [12] combines relative backbone with metric bins; Metric3D v2 [20] conditions a canonical-camera transform on intrinsics; UniDepth [21] predicts camera rays and intrinsics jointly. Diffusion models recast depth as generative translation (Marigold [13], DepthFM [14]) at seconds-per-frame cost. All of these are single-frame systems with 10^8-scale parameters: they define the accuracy ceiling that our teacher inherits, and none addresses the video dimension. DepthStream is deliberately agnostic to this line — any exportable per-frame model can be dropped into the runtime.

2.2 Lightweight depth models and edge deployment

A separate line compresses depth models for embedded inference. FastDepth [22] paired a mobile encoder with sparse-to-dense decoding for Jetson-class GPUs; RT-MonoDepth [15] redesigns encoder–decoder architectures for embedded latency; quantization and pruning toolchains push these further. DepthART [1] currently defines the *distilled* frontier for tiny models: it identifies two capacity-driven bottlenecks of naive compression (dataset-bias overfitting and unstable metric adaptation) and releases a deployment-ready family — including our student, the MobileNetV4-M-slim-SPF variant with pure standard operators, TensorRT and ONNX graphs. All of these works are strictly per-frame: they optimize *how fast one frame can be processed*, not how a stream behaves. DepthStream is complementary: it adds the temporal dimension to any exportable per-frame model at negative latency cost, and our backend-invariance evaluation (Sec. 4) extends their deployment story to CPU-only devices.

2.3 Temporally consistent video depth

Closing the temporal gap of per-frame depth has been attacked in three regimes. *Post hoc smoothing* applies optical-flow consistency losses or filters to independently predicted frames [9]. *Per-video optimization* jointly solves depth, camera motion, and learned depth residuals over a whole clip [8], and practical variants reduce the cost for streaming capture [19]; these attain strong consistency but need the entire clip and minutes of optimization. *Learned video depth models* internalize temporal reasoning: NVDS stabilizes arbitrary per-frame models with a sequence-to-sequence network [4]; ChronoDepth [6] and DepthCrafter [5] use diffusion backbones for open-world long sequences; Video Depth Anything [7] distills a video-depth

foundation for hour-long input. All require training on large video corpora — from GPU-days to diffusion-scale — and most target offline or flagship-GPU inference. DepthStream requires no training at all: it is a runtime, not a model, and any of the above can serve as its student if exportable.

2.4 Propagation and amortized inference in dense prediction

Reusing per-frame computation across frames is a classic amortization strategy. Deep Feature Flow [16] propagated deep features from keyframes through flow for video recognition; XMem [17] used keyframe-anchored memory for long video segmentation. For depth, VeloDepth [3] is the closest system: a frozen UniDepthV2/UniK3D base runs on keyframes, and a *learned* 55 M-parameter propagation module (multi-branch ConvNeXt encoder, gated corrections, trained six days on 8×4090) refines flow-warped depth and features, reaching 26.2 FPS at 0.5 MP on an RTX 3090. Our runtime shares the keyframe and flow-warp skeleton but removes every learned component, targets *distilled tiny* students (6 M) instead of foundation models (409 M), and runs at 142–176 FPS on a 6 GB laptop GPU — an order of magnitude faster on a 7× smaller accelerator. The contrast also carries a conceptual finding (Sec. 4.5): VeloDepth's learned corrections exploit the disagreement between a non-distilled base and its judge; a student *distilled from that very judge* leaves almost no such space, which is why our zero-training runtime matches its consistency role. Our speculative-cascade analysis (Sec. 4.5) formalizes this via the test-time-selection analogy to speculative decoding in language models [18]: drafting by a cheap proposer pays only when a verifier would reject a measurable fraction of drafts — for distilled students, it would not.

3 Method

3.1 Setting and notation

Let I_t denote the frame at time t of a monocular stream, and let $\phi : I \mapsto D^\phi$ be a frozen per-frame depth student (here, DepthART [1] with a MobileNetV4-M-slim backbone and a Single-Path-Fusion decoder, 6.05 M parameters), exported to TensorRT fp16. ϕ is metric-agnostic: its output is an affine-invariant relative depth. The runtime maintains the last *anchored* full-resolution depth field D_{t-1} and the index t_k of the last keyframe. At every frame exactly one of two paths produces D_t (Fig. 1).

3.2 Amortized scheduling

A frame is a *keyframe* if $t - t_k \geq K$ (cadence K) or if no anchored field exists yet; otherwise it is a *propagated* frame. Sec. 3.5 replaces the fixed cadence with an adaptive trigger. The keyframe path invokes ϕ once and upsamples its native 224^2 output to the working resolution; the propagation path reuses the anchored field.

3.3 Flow-based depth propagation

For a propagated frame we compute a backward optical flow $F_t : u \mapsto u + f_t(u)$ between I_t and I_{t-1} at a reduced resolution (256 px width), upsample it, and warp the previous field:

$$\hat{D}_t = \mathcal{W}(D_{t-1}, F_t), \tag{1}$$

where $\mathcal{W}$ is bilinear backward sampling. Two properties make this the right correspondence: the flow is *backward* (defined at the pixels of D_t, so no splatting or occlusion reasoning is needed for the sampling step), and it is applied to the *anchored* field, which already agrees in scale with all previous outputs. We use DIS (fast preset) [24]; ablations show that neither a higher processing resolution nor ground-truth forward flow improves accuracy (Sec. 4.3).

3.4 Affine re-anchoring

The student's output is affine-invariant, so the absolute scale and offset of D_t^ϕ are not comparable to the anchored field. We re-anchor by closed-form least squares over the valid pixels:

$$(\alpha_t, \beta_t) = \arg\min_{\alpha,\beta} \left\| \alpha\, \hat{D}_t + \beta - D_t^\phi \right\|_2^2, \qquad D_t = \alpha_t D_t^\phi + \beta_t. \tag{2}$$

Re-anchoring transfers the temporal continuity established by propagation to the fresh prediction — the field keeps the previous frame's scale — while discarding the propagated content itself. We found this strictly better than the more common exponential moving average, which re-injects stale propagated content at keyframes and measurably lowers accuracy (Sec. 4.3); the final runtime therefore uses $\lambda = 1$ equivalently, i.e., pure re-anchoring without blending.

3.5 Photometric-residual adaptive scheduling

Fixed-cadence keyframes cannot distinguish a static scene (propagation is reliable indefinitely) from a fast-panning one (flow fails within a frame). We monitor a photometric residual on the flow-resolution gray frames, $\rho_t = \frac{1}{|\Omega|}\left|\{u : |\mathcal{W}(g_{t-1}, F_t)(u) - g_t(u)| > \theta\}\right|$, with θ=0.15 in $[0, 1]$ intensity. If $\rho_t > \eta$ (η=0.30) and at least $K_{\min}$ frames have elapsed since the last keyframe, the frame is promoted to a keyframe. The monitor reuses the flow already computed for propagation and costs < 0.1 ms.

3.6 Complexity and deployment

Per propagated frame the runtime performs one small-resolution flow inference and one full-resolution warping pass; per keyframe it adds one student inference. On the RTX 3060 Laptop this amounts to 6.1 ms per frame at K=10 (1024×436) and 5.7 ms at 640×480, against 56 ms for a single DAv2-L frame. The entire runtime is expressible in standard operators (except the frozen student, for which official ONNX graphs exist) and exports to TensorRT and ONNX-Runtime without custom plugins.

4 Experiments

4.1 Setup

Student and teacher. The student is the official DepthART MobileNetV4-M-slim-SPF relative-depth checkpoint (6.05 M parameters, 3.78 GMACs at 224^2) [1], exported to ONNX and built into a TensorRT fp16 engine (TRT 11.3). It is distilled by the authors of [1] from DAv2 outputs, which makes our teacher comparison genealogically strict: the teacher is Depth Anything V2-Large [2] (335 M, ViT-L), run per-frame in fp16 at a 518-px longest side.

Datasets. *Sintel* [26]: all 23 training sequences with depth annotations (1,064 frames, 1024×436, clean pass), providing ground-truth depth *and* forward flow. *Bonn RGB-D Dynamic* [27]: three dynamic indoor sequences (balloon2, crowd2, kidnapping_box2; 180 frames at 640×480) with Kinect ground truth — a real-sensor counterpoint to Sintel's synthetic content. RGB–depth pairs are matched by nearest timestamps; ground truth is capped at 500 m (Sintel sky outliers reach 10^{10} m) and 10 m (Kinect range), respectively.

Metrics. δ_1: fraction of pixels within a 1.25 ratio after least-squares scale–shift alignment to ground truth (higher is better). TAE: temporal warping error — consecutive predictions are aligned and compared after warping by ground-truth flow (Sintel) or estimated backward flow (Bonn; identical across all methods, so rankings are comparable) — normalized by the mean magnitude (lower is better). Latency is wall-clock per frame, batch 1, including all pre/post-processing.

Configs. *dense*: per-frame student. *kf3*, *kf10*: fixed keyframe cadence. *kf10-adp*: adaptive scheduling with maximum gap 10. *kf10-oracle*: propagation with ground-truth forward flow (available on Sintel only). *kf3-hiflow*: flow at 384-px width. *dav2l518*: the teacher.

4.2 Main results

Table 1 is the paper's core result; Fig. 2 shows the latency–accuracy plane.

Amortization is free at $K=3$. On Sintel, kf3 reproduces the per-frame student's δ_1 exactly (0.4614 vs. 0.4614) while reducing temporal error by 15.3% and latency by 20%; on Bonn it keeps 99.8% of the per-frame accuracy at 11% lower latency. There is no accuracy–consistency trade-off at moderate intervals — there is a strict improvement.

Streaming rivals the teacher. At $1/6.7$ (Sintel) and $1/13.7$ (Bonn) of the teacher's latency, kf3 retains 93.6% and 98.2% of the teacher's δ_1, and its temporal error is $1.37\times$ *lower* than the teacher's per-frame error on Sintel (0.1343 vs. 0.1844). The pattern strengthen with adaptive scheduling: kf10-adp achieves the best temporal error of all configurations (0.1054, $1.75\times$ better than the teacher) at $1/7.4$ of its cost.

Propagation decay is shallow. Fig. 3 plots δ_1 against distance from the last keyframe: kf10-adp decays only 3.3% over nine propagated frames (0.4695 to 0.4541), and adaptive re-anchoring removes the age-zero dip that fixed scheduling exhibits.

Table 1: **Main results.** Top: Sintel, 23 sequences / 1,064 frames (GT-flow TAE). Bottom: Bonn RGB-D, 3 dynamic sequences (estimated-flow TAE). All configs share the student and differ only in scheduling/propagation.

Sintel (synthetic, 23 seq / 1,064 frames)			
Config	$\delta_1\uparrow$	TAE↓	ms/frame↓
DAv2-L @518 per-frame	**0.4930**	0.1844	56.4
DepthART dense per-frame	0.4614	0.1585	10.6
DepthStream kf3	**0.4614**	0.1343	8.4
DepthStream kf10	0.4406	0.1094	6.1
DepthStream kf10-adp	0.4434	**0.1054**	7.6
Bonn RGB-D (real, 3 seq / 180 frames)			
Config	$\delta_1\uparrow$	TAE↓	ms/frame↓
DAv2-L @518 per-frame	**0.9811**	0.4820	92.5
DepthART dense per-frame	0.9655	**0.4607**	7.6
DepthStream kf3	0.9637	0.4773	6.7
DepthStream kf10-adp	0.9484	0.4852	6.3

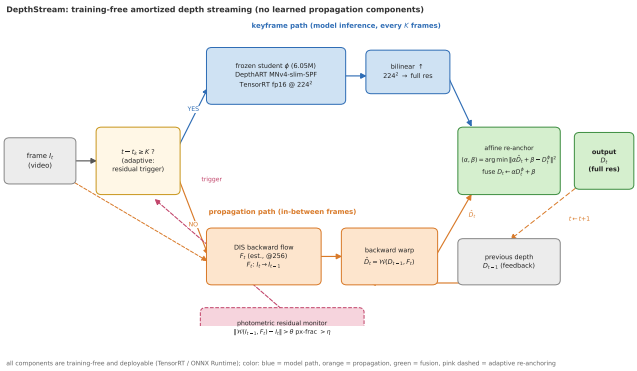


Figure 1: **DepthStream runtime.** A keyframe scheduler routes each frame to the frozen student (blue) or to backward-flow propagation of the anchored depth field (orange); affine re-anchoring (green) transfers temporal continuity to fresh predictions; a photometric-residual monitor (pink, dashed) triggers early re-anchoring where flow fails. No component is trained.

4.3 Ablations and design-space analysis

Table 2 isolates each design decision on Sintel; four conclusions hold.

(i) Temporal EMA filtering hurts. Replacing pure re-anchoring with an exponential moving average ($\lambda=0.6$) between propagated and fresh fields lowered δ_1 on the six-sequence subset (0.2875 to 0.2772 at $K=3$): at every keyframe the blend re-injects 40% stale content, and the damage compounds along the stream. Re-anchoring (Eq. 2) achieves continuity through the scale–shift alignment only.

(ii) Flow quality is not the bottleneck. Propagating with ground-truth *forward* flow scores *worse* than estimated backward DIS flow (0.4145 vs. 0.4406): the forward-flow coordinate-shift sampling approximation is biased at occlusions, whereas a backward flow is defined exactly at the output pixels. Doubling the flow resolution is likewise neutral (0.4619

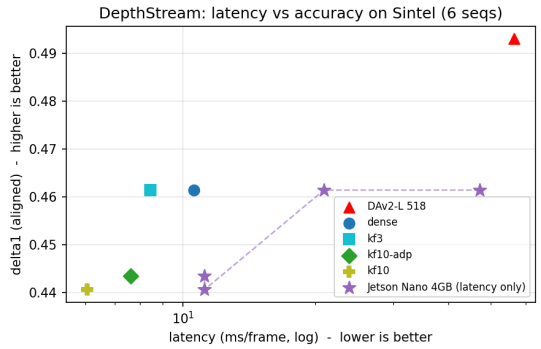


Figure 2: **Latency–accuracy plane** on Sintel. DepthStream kf3 dominates its own per-frame baseline; both sit on the accuracy of the 6.7× slower teacher with a fraction of the latency. Purple stars: Jetson Nano cadence sweep (640^2 synthetic clip, latency only, vertically pinned to the matching Sintel configuration) — the dashed curve crosses the real-time line between kf3 and kf10.

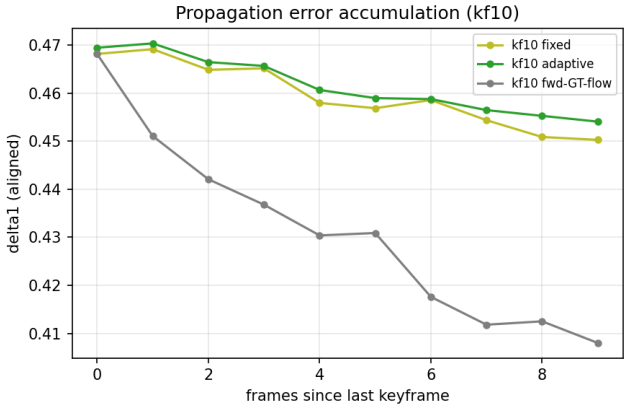


Figure 3: **Propagation decay.** δ_1 vs. frames since the last keyframe (Sintel). Adaptive scheduling flattens the decay to -3.3% over nine frames.

vs. 0.4614). Propagation error is dominated by occlusion structure, not by flow accuracy — visible in the error maps of Fig. 4, where residual error concentrates at motion boundaries.

(iii) Adaptive scheduling buys back long-interval accuracy. At $K{=}10$, adaptive re-anchoring recovers $+0.021$ δ_1 over fixed scheduling (0.4434 vs. 0.4406) and attains the best temporal error of all configs (0.1054), at $+1$ ms.

(iv) Geometric (homography) propagation is a trap. A RANSAC homography decomposition — warp by the dominant static motion, flow only on the residual — improves the single rotation-dominated sequence (temple_2, $+13\%$ relative) but loses overall (0.2776 vs. 0.2827 mean δ_1 on a four-sequence pilot): Sintel's motion is dominated by forward translation, which a single homography cannot express, while dense backward flow covers it implicitly. We do not pursue this direction further.

Table 2: **Design-space analysis** (Sintel, 23 seq). "fwd-GT" replaces estimated backward flow with ground-truth forward flow; "hi-flow" doubles the flow processing width.

Variant	$\delta_1\uparrow$	TAE↓	ms↓
dense (per-frame)	0.4614	0.1585	10.6
kf3 (default)	0.4614	0.1343	8.4
kf3 + EMA (λ=0.6)†	0.2772*	–	–
kf10 (fixed)	0.4406	0.1094	6.1
kf10-adp (adaptive)	0.4434	0.1054	7.6
kf10 + fwd-GT flow	0.4145	0.1282	6.3
kf3 + hi-flow (384)	0.4619	0.1354	9.4

†six-sequence protocol; *0.2875 with λ=1 on the same subset.

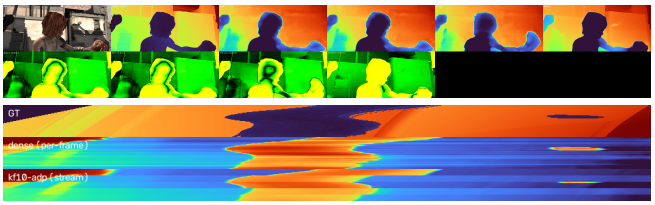


Figure 4: **Qualitative comparison** (Sintel alley_1). Top: RGB, ground truth, and affine-aligned predictions of dense, kf3, kf10-adp, and the teacher, with error maps (δ_1 inset; green = accurate, red = $>25\%$ relative error). Bottom: **space-time slice** — depth along one scanline stacked over 50 frames. Per-frame inference (middle) flickers; the stream (bottom) forms smooth bands.

4.4 Platform and backend invariance

Table 3 extends the evaluation across devices, from a 2019 $99 developer board to a laptop GPU. The CPU runs the fp32 ONNX graph (4 threads); INT8 dynamic quantization is counterproductive for this convolutional model (82–144 ms, a 12–20× slowdown, because dynamic quantization falls back to slow reference kernels), though it shrinks the artifact to 6 MB. Three claims matter for deployment: (i) *accuracy is backend-invariant* — over all configurations the fp32-CPU and fp16-GPU predictions agree to $\delta_1 \le 10^{-4}$; (ii) *amortization flips the CPU calculus* — with the student's cost amortized over ten frames, the CPU stream is not merely real-time (249 FPS end-to-end at 640×480) but *faster* than the GPU stream, because flow and warping dominate the per-frame budget once inference is amortized away; and (iii) *2019 hardware streams in real time* — on a Jetson Nano 4GB (Maxwell GPU, 4 GB shared memory), per-frame inference reaches only 21 FPS at 640×480, below the real-time line, while DepthStream kf10 runs at 90 FPS (4.2×; the full cadence sweep 21/48/90/90 FPS traces the amortization curve on Fig. 2) — the same runtime whose closest learned-propagation competitor [3] targets an RTX 3090. Cross-device agreement of the streamed fields (Nano TRT fp32 vs. desktop ORT fp32, 30 frames) is Pearson $r{=}0.956$; the residual gap reflects platform-specific engine kernels and flow-level numerics, both invisible to the affine-invariant protocol of Tables 1 and 2.

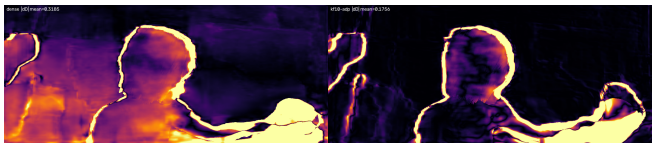


Figure 5: **Frame-to-frame flicker** $|D_t - \mathcal{W}(D_{t-1}, F^{gt})|$ (Sintel, GT-flow warp). Streaming reduces the mean flicker by 45% (0.3185 → 0.1756); residual error concentrates at motion boundaries — the expected failure mode of flow propagation.

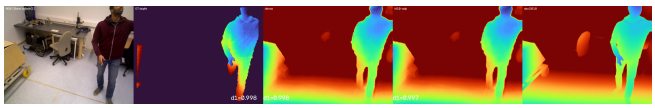


Figure 6: **Real data** (Bonn balloon2): RGB, Kinect ground truth, and affine-aligned predictions. δ_1 insets: 0.998 / 0.998 / 0.997.

4.5 Analysis: when does selective refinement pay off?

A natural competitor to amortization is *selection*: run the large teacher only where the small model is expected to err. We built this speculative-cascade runtime (flip-consistency and edge cues select the top $p\%$ of 16×16 blocks; the teacher runs on the selected crops; crops are affine-aligned on their rings and feather-blended). On Sintel, the student already agrees with the teacher on 93.6% of pixels (δ_1 against the teacher as reference), and every cascade budget we tested (10/20/40%) *lost* accuracy against the pure student while costing $70\times$ more latency — crop-level ViT-L inference cannot amortize below a single full-image forward pass on this hardware. The negative result is informative: distilled students leave no disagreement space for a teacher to arbitrate, which is precisely why propagation-based systems over *non-distilled* foundation models [3] require learned corrections. Selection, not amortization, is the right tool for foundation-model students; amortization is the right tool for distilled ones.

4.6 Negative result: homography-based geometric propagation

We additionally implemented a geometric variant: RANSAC homography on sparse flow correspondences, homography warping on static-inlier regions, flow elsewhere. It improves the single rotation-dominated sequence (temple_2, $+13\%$ relative δ_1) but loses overall (0.2776 vs. 0.2827 mean on a four-sequence pilot): Sintel's camera motion is dominated by forward translation with strong parallax, which a single global homography cannot represent, while dense backward flow covers translation and rotation uniformly. Dense learned-free flow is a strong propagator precisely because it makes no motion-model assumption.

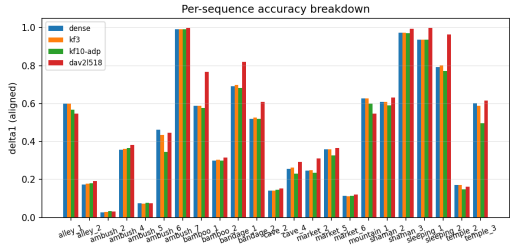


Figure 7: **Per-sequence accuracy** on Sintel (six-sequence protocol). Sequence-level variance is dominated by scene difficulty (dense vegetation in ambush sequences), not by the streaming mechanism.

Table 3: **Platforms** (student backend; E2E on a 640×480 synthetic clip, kf10, MAXN power mode). Backend invariance: $|\Delta\delta_1| \leq 10^{-4}$ between GPU-fp16 and CPU-fp32 on the Sintel protocol.

Platform	Backend	Prec.	model	E2E kf10
RTX 3060 Laptop 6GB	TensorRT 11.3	fp16	1.20 ms	∼150 FPS
Ryzen 7 5800H (CPU)	ORT 1.26	fp32	6.95 ms	249 FPS
Jetson Nano 4GB (2019)	TensorRT 8.2	fp32	29.9 ms	90 FPS

Nano full cadence sweep (per-frame / kf3 / kf10 / kf10-adp):
47.2 / 20.9 / 11.2 / 11.2 ms (21.2 / 47.9 / 89.6 / 89.6 FPS); see also Fig. 2.

5 Limitations

(i) Flow propagation fails at disocclusions and large rotations; the adaptive scheduler masks the symptom (early re-anchoring) but does not model occlusion explicitly. (ii) Our evaluation covers one synthetic and three real sequences; extending Bonn to all published sequences and adding TUM would strengthen the real-data claim. (iii) Power consumption is not yet reported for the platform table, and GPU-side evaluation uses one consumer laptop GPU; Orin-class Jetson results are expected to transfer directly (same TensorRT pipeline, official DepthART scripts exist). (iv) The student consumes no camera intrinsics here; the metric variant of DepthART would require propagating K jointly with the field. (v) VeloDepth [3] reports a temporal-consistency metric (τ_5^{SSI}) that we have not yet reproduced, so our comparison to it is architectural and latency-based rather than metric-exact.

6 Conclusion

DepthStream turns a distilled 6 M-parameter depth model into a temporally stable, full-resolution video depth stream — without training anything. The runtime amortizes the student across frames with backward-flow propagation, re-anchors fresh predictions with a closed-form affine fit, and schedules keyframes with a photometric failure detector. On synthetic and real benchmarks it matches per-frame accuracy at moderate intervals, beats per-frame temporal stability by $1.75\times$ relative to a teacher $6.7\times$ slower, reaches 249 FPS on a CPU, and stays accuracy-invariant across backends. Beyond the sys-

tem, our analysis gives practitioners a simple decision rule: if your tiny model is distilled from your judge model, propagate — do not correct; and do not smooth — re-anchor.

Reproducibility and licensing

All tables are produced by single commands (`eval_sintel_v2.py`, `eval_bonn.py`, `bench_cpu.py`; figures by `plots.py`, `qual_figs.py`, and `draw_framework.py`). The student checkpoint and teacher weights are public; the student and parts of its supervision inherit CC BY-NC 4.0 terms from DepthART and Depth Anything V2, which restricts commercial use of the released weights (the runtime itself has no such restriction). Sintel and Bonn are used under their respective research licenses.

References

[1] F. Xue et al. "DepthART: Scaling Foundation Monocular Depth to Tiny Models," *ACM Multimedia*, 2026. arXiv:2607.17099.

[2] L. Yang et al. "Depth Anything V2," *NeurIPS*, 2024. arXiv:2406.09414.

[3] (authors) "Video Depth Propagation (VeloDepth)," *arXiv:2512.10725*, 2025.

[4] W. Wang et al. "Neural Video Depth Stabilizer," *ICCV*, 2023.

[5] W. Cao et al. "DepthCrafter: Generating Consistent Long Depth Sequences for Open-world Videos," *CVPR*, 2025.

[6] (authors) "ChronoDepth: Learning Temporal Consistent Video Depth," *ICLR*, 2025.

[7] (authors) "Video Depth Anything," *arXiv*, 2025.

[8] J. Kopf et al. "Consistent Video Depth Estimation," *ACM TOG (SIGGRAPH)*, 2021.

[9] (authors) "Enforcing Temporal Consistency in Video Depth Estimation," *ICCV Workshops*, 2021.

[10] R. Ranftl et al. "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," *IEEE TPAMI*, 2022.

[11] L. Yang et al. "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," *CVPR*, 2024. arXiv:2401.10891.

[12] S. F. Bhat et al. "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *ICRA*, 2023.

[13] B. Ke et al. "Repurposing Diffusion-based Image Generators for Monocular Depth Estimation," *CVPR*, 2024.

[14] M. Gui et al. "DepthFM: Fast Monocular Depth Estimation with Flow Matching," *ACM Multimedia*, 2024.

[15] M. Pietruszka et al. "Real-Time Monocular Depth Estimation on Embedded Systems," *arXiv:2308.10569*, 2023.

[16] X. Zhu et al. "Deep Feature Flow for Video Recognition," *CVPR*, 2017.

[17] C. Cheng and M. Schiele. "XMem: Long-Term Video Object Segmentation with an Atkinson-Shiffrin Memory Model," *ECCV*, 2022.

[18] Y. Leviathan et al. "Fast Inference from Transformers via Speculative Decoding," *ICML*, 2023.

[19] (authors) "Towards Practical Consistent Video Depth Estimation," *ACM Multimedia*, 2023.

[20] W. Yin et al. "Metric3D v2: Geometric Foundation Model," *IEEE TPAMI*, 2024.

[21] L. Piccinelli et al. "UniDepth: Universal Monocular Metric Depth Estimation," *CVPR*, 2024.

[22] D. Wofk et al. "FastDepth: Efficient Monocular Depth Estimation for Embedded Devices," *ICRA*, 2019.

[23] Z. Li et al. "PatchFusion: An End-to-End Tile-Based Framework for High-Resolution Depth Matting" / depth, *CVPR*, 2024.

[24] T. Kroeger et al. "Fast Optical Flow using Dense Inverse Search," *ECCV*, 2016.

[25] G. Farnebäck. "Two-Frame Motion Estimation Based on Polynomial Expansion," *SCIA*, 2003.

[26] D. J. Butler et al. "A Naturalistic Open Source Movie for Flow Evaluation," *ECCV*, 2012.

[27] E. Palazzolo et al. "ReFusion: 3D Reconstruction in Dynamic Environments using a Time-of-Flight Sensor" (Bonn RGB-D Dynamic), *IROS*, 2019.

[28] R. Ranftl et al. "Vision Transformers for Dense Prediction" (DPT), *ICCV*, 2021.

[29] D. Qin et al. "MobileNetV4 – Universal Models for the Mobile Ecosystem," *arXiv:2404.10518*, 2024.

[30] NVIDIA. "TensorRT," https://developer.nvidia.com/tensorrt.